Appendix B: Data source excerpt

data_source.py

```
import configparser
import os
import re
import threading
from datetime import datetime
from pathlib import Path

from preprocessing.pingstation_raw_decoder import (
    PingstationRawCaptureLogger,
    PingstationRawDecoder,
    PingstationStreamReceiver,
)
from runtime_paths import app_dir, resource_dir


# ---------------------------------------------------------------------------
# Config-file support
# A file called  ADSBTracker.cfg  placed next to the exe (or in the project
# root during development) can override any of the env-var defaults below.
# If the file does not exist it is created automatically with full comments.
# ---------------------------------------------------------------------------
_CFG_PATH = app_dir() / "ADSBTracker.cfg"
_CFG_TEMPLATE = """\
; ADS-B Tracker configuration file
; Edit this file to switch between simulation and live mode.
; Restart the application after making changes.
;
; ------------------------------------------------------------
; MODE SELECTION
; ------------------------------------------------------------
[mode]
; Set to false to receive LIVE data from the PingStation device.
; Set to true  to replay the bundled sample log file (no hardware needed).
simulation_mode = true

; ------------------------------------------------------------
; LIVE MODE  (only used when simulation_mode = false)
; ------------------------------------------------------------
[live]
; The PingStation device must be configured (via its web interface) to send
; its RAW data stream to THIS PC's IP address on the port below.
; The tracker opens a TCP server socket and waits for the PingStation to
; connect to it.
listen_host = 0.0.0.0
listen_port = 30003
; Seconds before an aircraft that stops broadcasting is removed from the map.
stale_seconds = 45
; Save a copy of every received byte to  logs/sessions/<session>/
capture_enabled = true

; ------------------------------------------------------------
; SIMULATION MODE  (only used when simulation_mode = true)
; ------------------------------------------------------------
[simulation]
; Path to the hex log file to replay (leave blank to use the bundled sample).
source_file =
; How many log entries to process per refresh cycle.
replay_batch = 20
; Loop the replay file indefinitely (true) or stop at end (false).
replay_loop = false
; Seconds before a replayed aircraft is considered stale and removed.
stale_seconds = 12
"""

def _load_cfg():
    """Return a ConfigParser loaded from ADSBTracker.cfg, creating it if absent."""
    if not _CFG_PATH.exists():
        try:
            _CFG_PATH.write_text(_CFG_TEMPLATE, encoding="utf-8")
            print(f"[Config] Created default configuration file: {_CFG_PATH}")
        except OSError as exc:
            print(f"[Config] Could not write config file: {exc}")
    cfg = configparser.ConfigParser()
    cfg.read(str(_CFG_PATH), encoding="utf-8")
    return cfg

def _cfg_get(cfg, section, key, fallback_env, default):
    """Read from config file, fall back to env var, then to hard-coded default."""
    env_val = os.getenv(fallback_env)
    if env_val is not None:
        return env_val          # env var always wins over file
    try:
        return cfg.get(section, key)
    except (configparser.NoSectionError, configparser.NoOptionError):
        return default

_cfg = _load_cfg()


DEFAULT_SIMULATION_LOG_FILE = str(
    resource_dir() / "pingstation_data_log" / "pingstation_data_log" / "HEX DATA_H.txt"
)

_SESSION_STAMP = datetime.utcnow().strftime("%Y%m%d_%H%M%S")
SESSION_DIR = Path(
    os.getenv(
        "PINGSTATION_SESSION_DIR",
        str(app_dir() / "logs" / "sessions" / f"session_{_SESSION_STAMP}"),
    )
)
RAW_HEX_DIR = SESSION_DIR / "hex_data"
RAW_ASCII_DIR = SESSION_DIR / "ascii_data"
ICAO_EXCEL_DIR = SESSION_DIR / "icao_excel"
RAW_HEX_DIR.mkdir(parents=True, exist_ok=True)
```